

PART B — Task 2 (Peterson / Filter Lock for 3 processes)

4) Explain the algorithm clearly

Task 2 moves beyond Dekker. Dekker is only for 2 processes. For more than 2, we need a generalisation. The brief provides an algorithm based on `level[i]` and `last_to_enter[i]`.

Intuitively, each process climbs through a sequence of ‘gates’ or ‘levels’ before entering the critical section. With 3 processes, there are two gates: level 0 and level 1. A process sets its level, records itself as the last to enter that level, and then waits while:

- it is still the last entrant at that level, and
- some other process is at the same or higher level.

That waiting condition is what enforces ordering.

By the time a process reaches the final level, it can enter the critical section, and crucially, only one can be there at a time.

When it exits the critical section, it releases the lock by setting `level[i] = -1`.”

your modelling assumption have to be clear (because the pseudocode in the brief uses `FOR l = 0 TO n-1` but `last_to_enter[n-1]` only exists for `n-1` entries):

“For `n=3`, we interpret the algorithm as having gate levels `l = 0` and `l = 1` (two gates), then the critical section. That matches the `last_to_enter[0..1]` structure in the brief.”

5) Step-by-step CSPM creation for Task 2

Step 1 — Choose observable events

“We again use `enterCS.i` and `exitCS.i` as observable events for mutual exclusion. These are the events our safety spec will constrain.”

Step 2 — Model shared registers

We model each shared register as a state process:

- `level[0]`, `level[1]`, `level[2]` each store an Int
- `last_to_enter[0]`, `last_to_enter[1]` each store an Int”

Step 3 — Implement the gate-wait condition

“In CSP we implement the ‘while’ loop as repeated reads.

A process reads `last_to_enter[1]` and reads the other processes’ levels.

If the condition is true, it recurses (spins).

If the condition is false, it proceeds to the next level.”

Step 4 — Compose the three processes with shared memory

“All 3 processes run in parallel and synchronise on the shared memory read/write events.”

6) What exactly to check in FDR for Task 2 (as per brief)

“The brief requires you to prove:

1. Mutual exclusion is correct.
2. Progress is not blocked by the algorithm (assuming the critical sections do not block).

In FDR we demonstrate this by:

- A **traces refinement** safety check for mutual exclusion
- plus showing the system is **deadlock free**
- and **divergence free** (bounded waiting / no livelock).”

So the checks are:

- `assert SYSTEM3 [T= SPEC_CS3]`
- `assert SYSTEM3 :[deadlock free]`
- `assert SYSTEM3 :[divergence free]`

✓ FULLY WORKED MODEL SOLUTION — Task 2 (3-process Peterson/Filter lock)

Save as: `task2_username.csp`

```
-- =====  
-- 6CM504 Exercise 6 - Task 2  
-- 3-process Peterson-style (Filter lock) mutual exclusion
```

```

-- Based on level[i] and last_to_enter[l] from the brief.
-- =====

-- Observable critical section events
channel enterCS, exitCS : {0,1,2}

-- Shared registers: level[i] for i in {0,1,2}, values in {-1,0,1}
-- We will use -1 for "not competing"
channel wlevel : {0,1,2}.Int
channel rlevel : {0,1,2}.Int

-- last_to_enter[l] for l in {0,1}, values in {0,1,2}
channel wlast : {0,1}.Int
channel rlast : {0,1}.Int

-- -----
-- Shared register processes
-- -----

LEVEL(i, val) =
    wlevel.i?x -> LEVEL(i, x)
    [] rlevel.i!val -> LEVEL(i, val)

LAST(l, val) =
    wlast.l?x -> LAST(l, x)
    [] rlast.l!val -> LAST(l, val)

-- -----
-- Helper: other ids for n=3
-- -----

others(0) = <1,2>
others(1) = <0,2>
others(2) = <0,1>

fst(<a,b>) = a
snd(<a,b>) = b

-- -----
-- Gate waiting condition for level l (l in {0,1})
-- While (last_to_enter[l] == i) AND (exists k != i with level[k] >=
l) do spin.
-- Implemented by repeated reads.
-- -----

WAIT_GATE(i, l) =
    rlast.l?victim ->
    let os = others(i) within
    rlevel.(fst(os))?l1 ->
    rlevel.(snd(os))?l2 ->
    if (victim == i) and ((l1 >= 1) or (l2 >= 1)) then
        WAIT_GATE(i, l)
    else
        SKIP

-- -----
-- Process i: for n=3 there are two gates l=0 and l=1, then CS.
-- On exit: level[i] := -1
-- -----

PROC(i) =
    -- Gate 0

```

```

wlevel.i!0 ->
wlast.0!i ->
WAIT_GATE(i, 0) ;
-- Gate 1
wlevel.i!1 ->
wlast.1!i ->
WAIT_GATE(i, 1) ;
-- Critical section
enterCS.i -> exitCS.i ->
-- Release
wlevel.i!(-1) ->
PROC(i)

-- -----
-- System composition with shared memory
-- Initial: all level = -1 (nobody competing), last_to_enter
arbitrary (set to 0)
-- -----

MEMORY =
  (LEVEL(0, -1) [|{|wlevel.1, rlevel.1, wlevel.2, rlevel.2, wlast,
rlast|}|])
  LEVEL(1, -1) [|{|wlevel.2, rlevel.2, wlast, rlast|}|])
  LEVEL(2, -1) [|{|wlast, rlast|}|])
  LAST(0, 0) [|{|wlast.1, rlast.1|}|])
  LAST(1, 0))

SHARED = {| wlevel.0, rlevel.0, wlevel.1, rlevel.1, wlevel.2,
rlevel.2, wlast.0, rlast.0, wlast.1, rlast.1 |}

SYSTEM3 =
  (PROC(0) [|SHARED|] PROC(1) [|SHARED|] PROC(2))
  [|SHARED|]
  MEMORY

-- -----
-- Mutual exclusion spec: cannot see enterCS of one process while
another is "inside"
-- i.e., enterCS.i must be followed by exitCS.i before any enterCS.j
(j!=i)
-- -----

SPEC_CS3 =
  enterCS.0 -> exitCS.0 -> SPEC_CS3
  [] enterCS.1 -> exitCS.1 -> SPEC_CS3
  [] enterCS.2 -> exitCS.2 -> SPEC_CS3

-- -----
-- Checks required by the brief:
-- 1) mutual exclusion (safety) and 2) no blocked progress (deadlock-
free) and bounded waiting (divergence-free)
-- -----

assert SYSTEM3 [T= SPEC_CS3]
assert SYSTEM3 :[deadlock free]
assert SYSTEM3 :[divergence free]

```
